



ISLAMIC UNIVERSITY OF TECHNOLOGY



Phase C, D, E (SW) Report

EEE 4308

Digital Electronics Lab Project

Project Name:

Digital Snake Ladder Game

Group Name:

NSB

Group Members:

	Student ID	Member Name
1.	220021346	Musfique Ullah Khan Siam
2.	220021348	Fasahat Labib
3.	220021332	Fardin Ahmed Chowdhury
4.	220021345	MD.Mahir Rahman
5.	220021344	Kazi Mokabbir Hossain
6.	220021327	Kazi Adit Raiyan

Supervisor:

Mr. A. K. M. Rakib

Lecturer, Department of Electrical and Electronic Engineering,
Islamic University of Technology.

Objective:

The goal of this project is to implement the snake ladder game digitally on both software and hardware using combinational and sequential logic circuits.

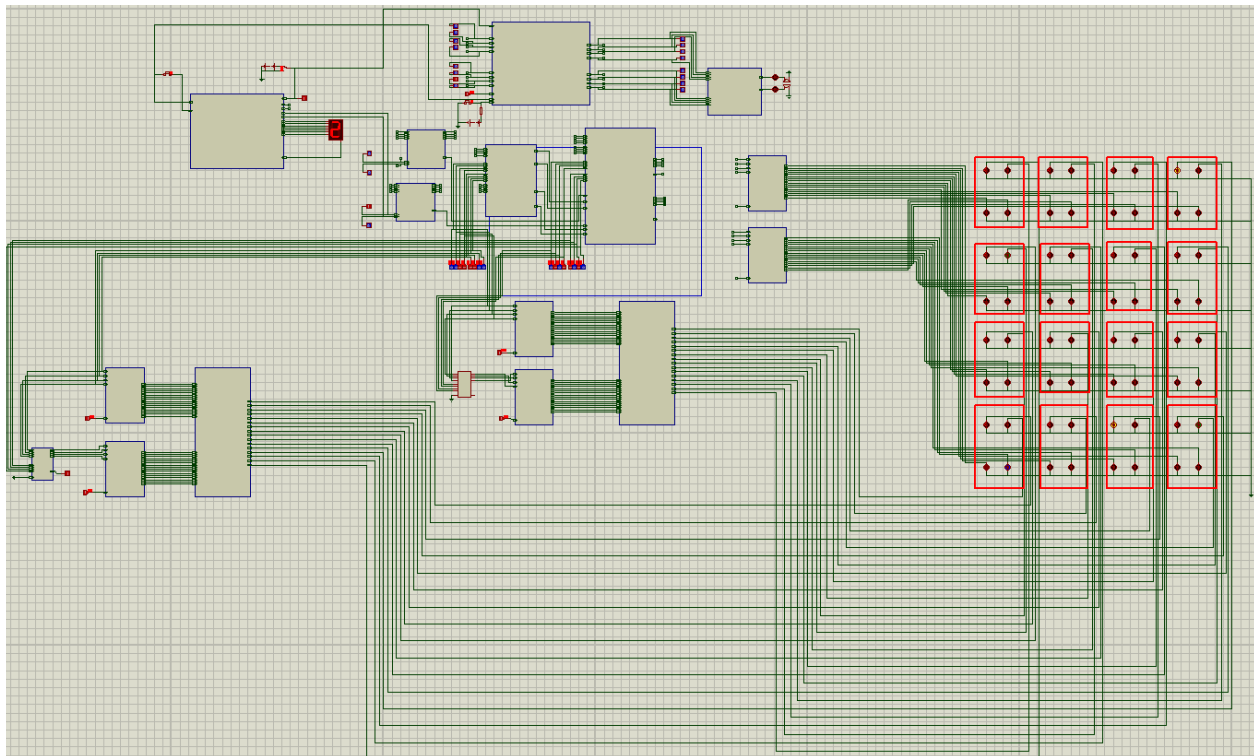
Brief Description of the Project:

The snake ladder game is a classic board game played between two or more players. This is a game of chance, meaning the outcome of the game is a random event determined by the roll of a dice. The goal of this game is to be the first to reach the ending square, usually 100, from the beginning square, 1. While snakes “eat” the player if met and send them back to a lower square, and ladders promote the player to a higher square.

This project aims to design the game on a 4x4 LED grid. There will be a total of 16 positions, denoted with 0 through 15 in logic. Each position is visually represented by 4 LEDs. The first two for the players and the latter two for the snake and the ladder. We have chosen red and blue for player 1 and 2 respectively, yellow for the snake, and green for the ladder. A roll of dice, simulated by the dice module to generate a pseudo-random number from 1 through 3 shifts the players’ position by that number. If a snake or a ladder is met, the player is demoted or promoted. The new 4-bit position of the player is stored in the memory and a player selector module is used to make players take turns to roll the dice. The player reaching the position 15 first wins the game. A win is determined by the winning module that compares the player’s current position to the winning condition. The win is then visually represented with buzzers and LEDs to notify the other player about the game result. There is also a reset module that can start the game all over from any position.

Circuit Diagram:

Here is the circuit prepared upon completion of phases C, D, E. This diagram consists of the entire snake and ladder game schematic on the Proteus 8 software. The board module, the snake, the ladder, and their decoders, player position decoder, snake and ladder logic module, dice module, memory module, reset module, player selector module, winner indicator module are present here. The modules are shown as subcircuits.

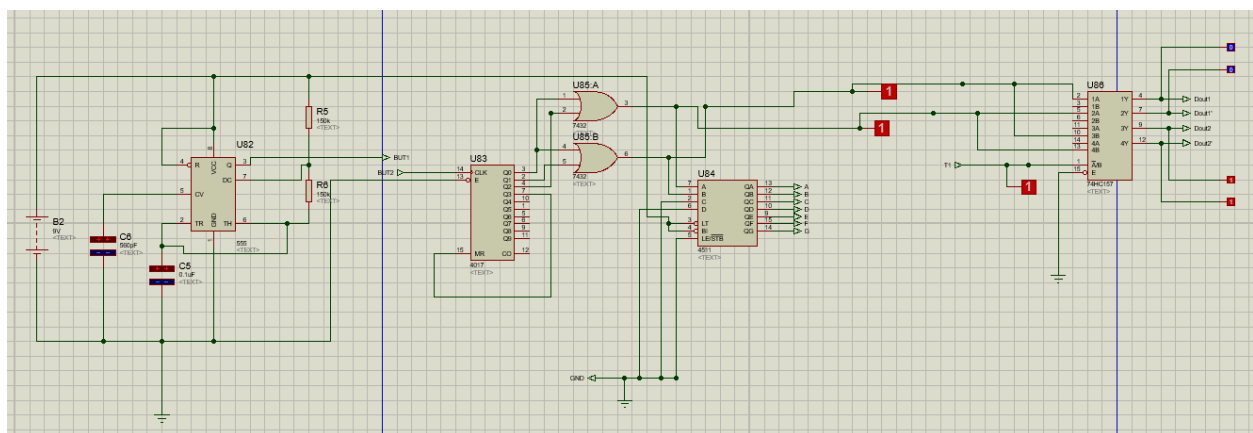
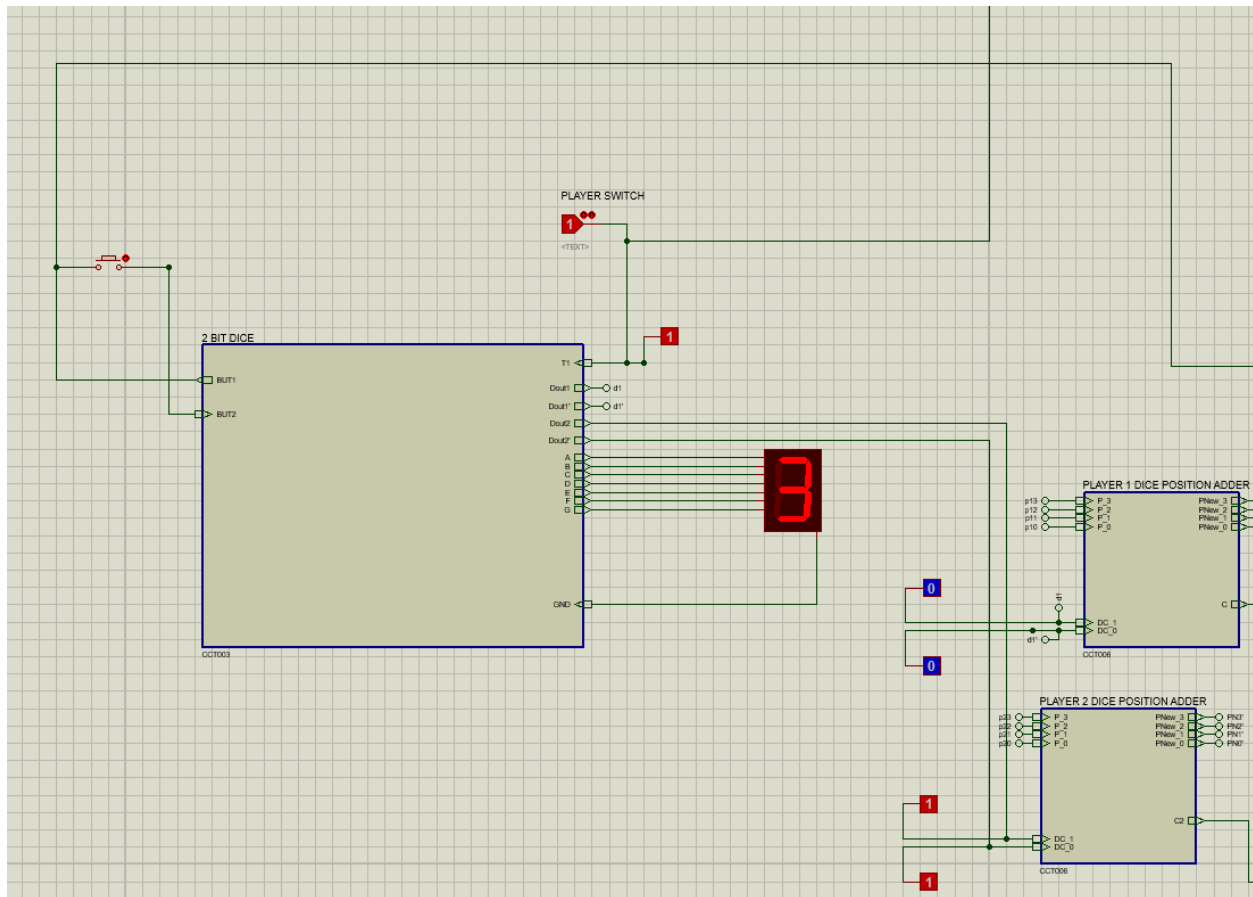


Now we shall discuss the modules that were asked in the phases C, D, and E.

Dice Module:

This module generates a 2-bit pseudo-random number between 1 and 3. The Dice Module uses a high-speed counter to generate a random number between 1 and 3 and shows the result on a 7-segment display, simulating a

dice roll. The quick number cycling and the player's timing while releasing the dice button provides the unpredictability factor.



The Dice Module is created using 3 components, they are a 555 timer , a clock counter and a seven segment BCD decoder and a seven segment display to see the output from the dice. There is a button connected in the middle of the 555 timer and the clock counter.

555 timer:

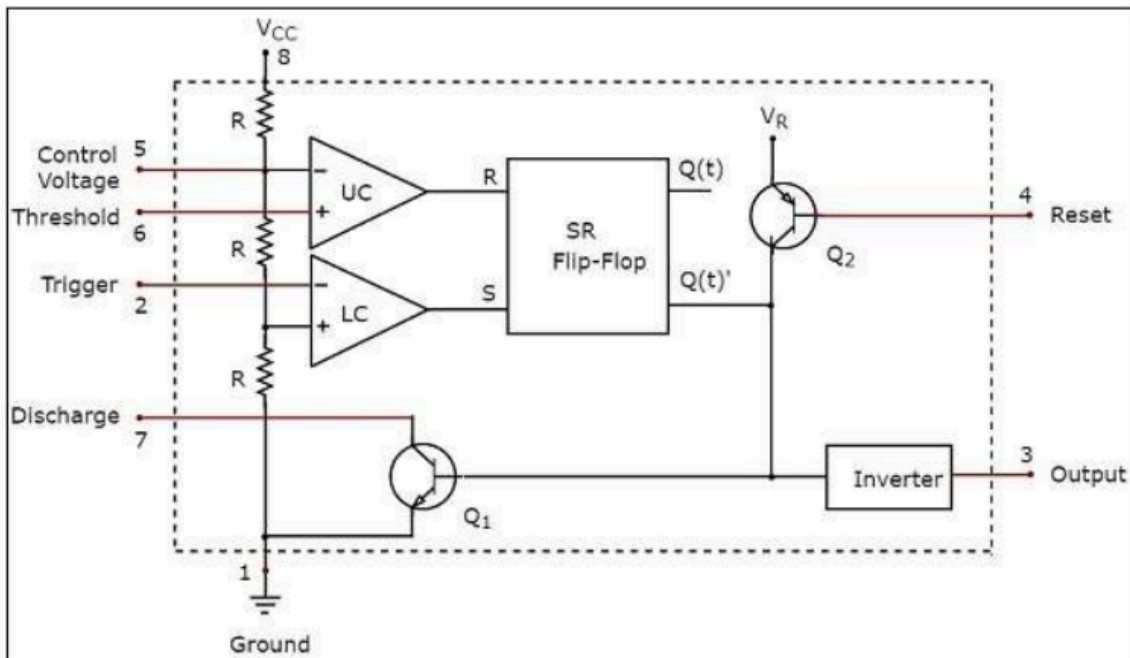
The 555 timer IC is an integral part of any electronics projects. Be it a simple project involving a single 8-bit microcontroller and some peripherals or a complex one involving a system on chips (SoCs), a 555 timer is involved. These provide time delays, as an oscillator and as a flip-flop element among other applications.

Here the 555 timer is used as a multivibrator to create a clock pulse that is fed into the clock counter. 555 timer changes the output from 0 to 1 and 1 to 0 in a fixed frequency that is dependent on the value of resistors and capacitors, here we used 150k and resistors and a capacitor with 560pF and 0.1 uF, so that we gate a high frequency clock pulse.

Functional Diagram

The pictorial representation showing the internal details of a 555 Timer is known as functional diagram.

The **functional diagram** of 555 Timer IC is shown in the following figure –



We used the following formula

Monostable
The signal duration of the output can be modeled as Time in seconds (T) equal to the constant 1.1 multiplied by the resistance R measured in ohms (Ω) multiplied by the capacitance value C measured in Farads (F).

$$T = 1.1R_1C_1$$

So, for a circuit with 10,000 (10k) ohms (Ω) of a resistance and a capacitance value of 1,000 microfarads (1000 μF), the total length of the output signal would be as follows

$$T = 1.1 \times 10,000\Omega \times 0.001F = 11s$$

Astable
The continuous output signal in Astable mode allows you to determine both the frequency in Hertz (Hz), how long it takes the signal to repeat and the duration of time the signal is in the high and low states. All three values that would define the output signal can be found using the values of R1 and R2 measured in ohms (Ω) and the capacitance value C1 measured in Farads (F).

$$T_h = 0.693(R_1 + R_2)C_1$$

$$T_l = 0.693R_2C_1$$

$$f = \frac{1.44}{(R_1 + 2R_2)C_1}$$

So, for a circuit with resistors of 10,000 (10k) and 15,000 (15k) ohms (Ω) of a resistance and a capacitance value of 10 microfarads (10 μF), the output signal would have the following characteristics

Time High

$$T_h = 0.693(10,000\Omega + 15,000\Omega)0.00001F = 0.1732s$$

Time Low

$$T_l = 0.693(15,000\Omega \times 0.00001F) = 0.1039s$$

Frequency

$$f = \frac{1.44}{(10,000\Omega + 2 \times 15,000\Omega)0.00001F} = 3.6Hz$$

Calculation of the 555 timer:

CHOOSE CONFIGURATION

Monostable

Astable

R₁ RESISTOR VALUE

150

kΩ

R₂ RESISTOR VALUE

150

kΩ

C₁ CAPACITANCE VALUE

0.1

μF

FORMULAS

$T_h = 0.693(R_1 + R_2)C_1$

$T_l = 0.693R_2C_1$

$f = \frac{1.44}{(R_1 + 2R_2)C_1}$

TIME HIGH

20.79

mS

TIME LOW

10.395

mS

FREQUENCY

32

Hz

V_{cc}

R₁

7 (DISCHARGE)

R₂

6 (THRESHOLD)

2 (TRIGGER)

C₁

1 (GND)

555

8 (VCC)

4 (RESET)

3 (OUTPUT)

TIME HIGH

20.79 milliseconds

TIME LOW

10.395 milliseconds

Search Catalog

Clear Selection

Clock counter:

A Counter is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relation to a clock signal. Counters are used in digital electronics for counting purposes, they can count specific events happening in the circuit. For example, in the UP counter a counter increases count for every rising edge of clock. Not only counting, a counter can follow a certain sequence based on our design like any random sequence 0,1,3,2... They can also be designed with the help of flip flops. They are used as frequency dividers where the frequency of a given pulse waveform is divided. Counters are sequential circuits that count the number of pulses can be either in binary code or BCD form. The main properties of a counter are timing , sequencing , and counting. Counter works in two modes : Up and down counters.

Decade Counter:

A decade counter counts ten different states and then reset to its initial states. A simple decade counter will count from 0 to 9 but we can also make the decade counters which can go through any ten states between 0 to 15(for 4 bit counters).

In the following circuit we used a decade counter to count the values. As we only need a 2 bit dice so the value would be 1,2, or 3 (01,10,11), so we used only the first 3 pins for the count then we selected the 4th(Q3) pin to reset the counter. SO that it quickly just counts 1 to 2 to 3 and then to 1 again and it works like that.

And we connected the clock pin to the input frequency that we get from the 555 timer and connected a button in the middle.

So, The module rapidly cycles through the numbers 1 through 3, which are displayed on a 7-segment display. The dice button initiates the counter, and when the button is released, it stops and locks in the current number determined by the dice roll.

A **7-segment display** linked to a BCD to 7-segment decoder is used to show the output.

The dice button is pressed to start the counter, which quickly cycles through the numbers until the button is released, at which time the held and displayed number is revealed.

There are 2 OR gates because when the counter is at 1 (1 0 0) the output in the seven segment decoder must be (0 1) when the counter is at 2 (0 1 0) the 7 segment must show the value two (1 0) and likewise when the counter is at 3 (0 0 1) the display should show 3 (1 1) for making this circuit we used 2 OR gates as if one of the input of a or gate is 1 the output should be one . So Q0, Q1 connected with or-gate is connected to the first input of the 7-seg display and Q0 and Q2 are connected to another or gate to the second input of the 7-seg display.

Memory Module:

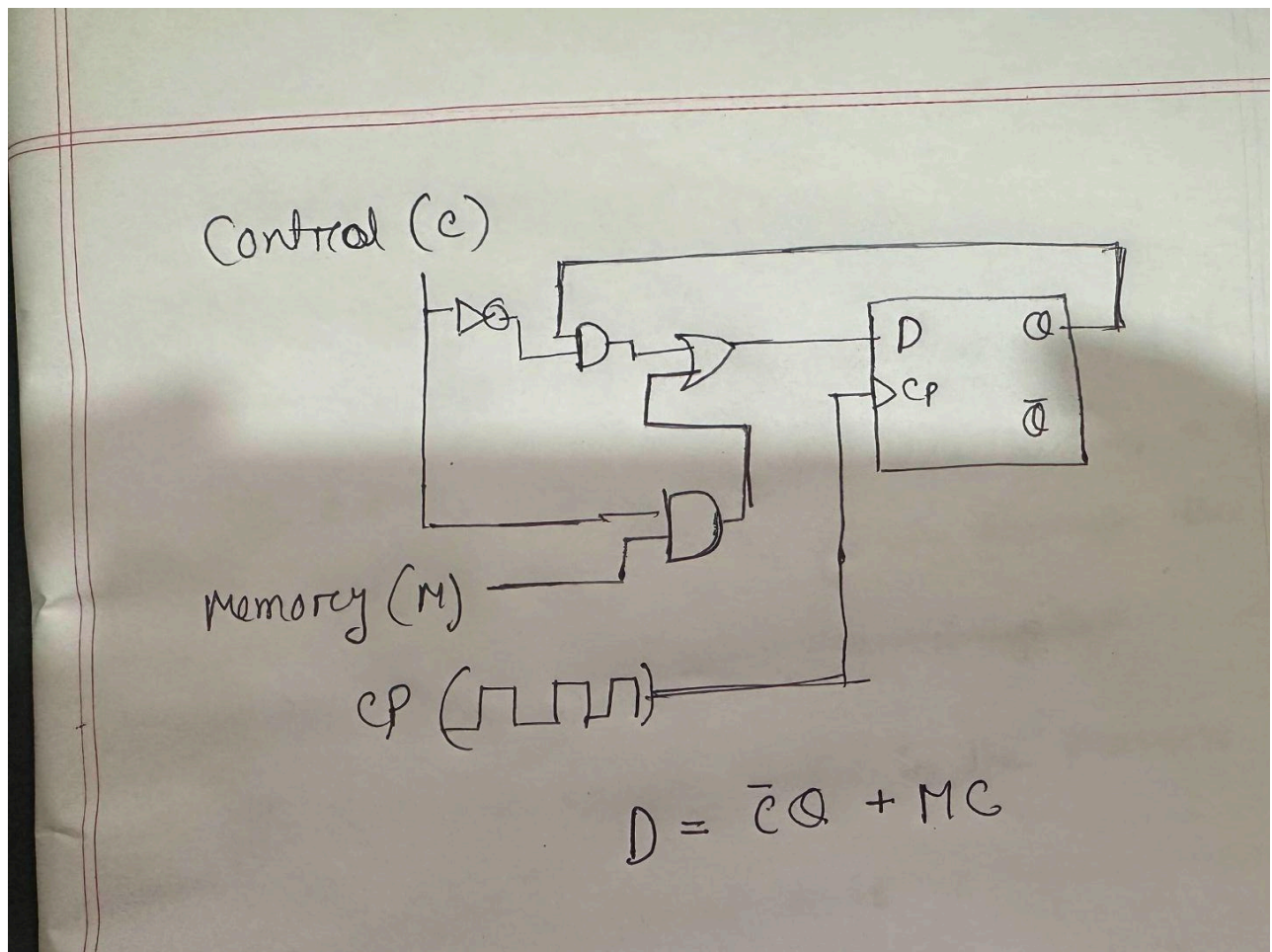
<u>INPUT</u>		<u>Present State</u>	<u>Next State</u>	<u>D flip flop input</u>
<u>C</u>	<u>M</u>	<u>Q(t)</u>	<u>Q(t+1)</u>	<u>D</u>
<u>0</u>	<u>X</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>0</u>	<u>X</u>	<u>1</u>	<u>1</u>	<u>1</u>
<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>
<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>

K map

<u>CM</u>	<u>Q</u>	
<u>00</u>	<u>0</u>	<u>1</u>
<u>01</u>	<u>x</u>	<u>x</u>
<u>11</u>	<u>1</u>	<u>1</u>
<u>10</u>	<u>0</u>	<u>0</u>

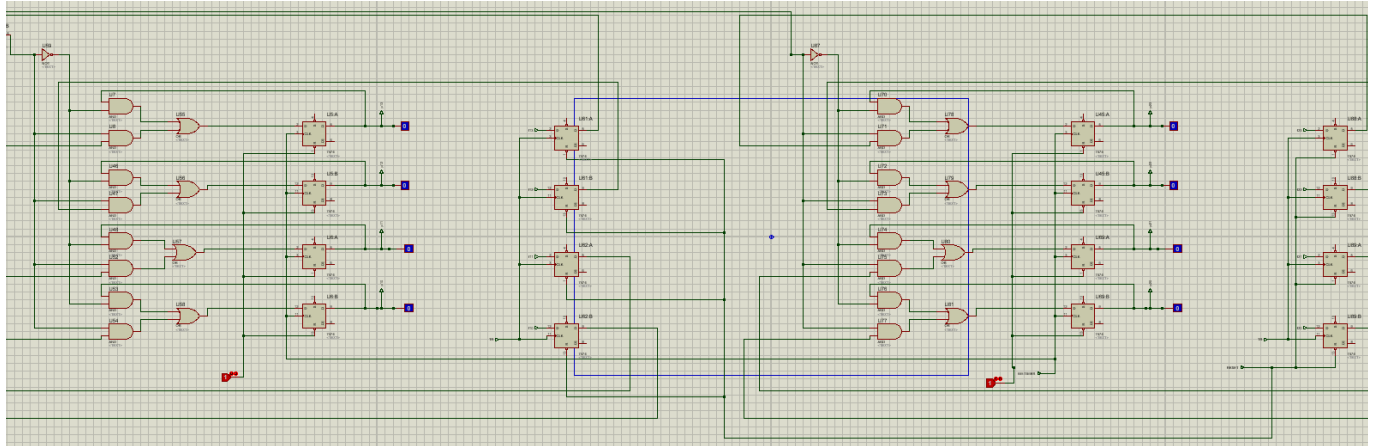
$$D = C'Q + MC$$

Circuit Diagram:



Description of Memory,PIPO Shift register, Proteus implementation,Trigger:

Memory: Memory is a place for storing data and recovering it whenever the data is needed. In this project of snake ladder game we built a memory which helps us to store the dice roll results,positions of the players and maintains the positions of snakes and ladders.

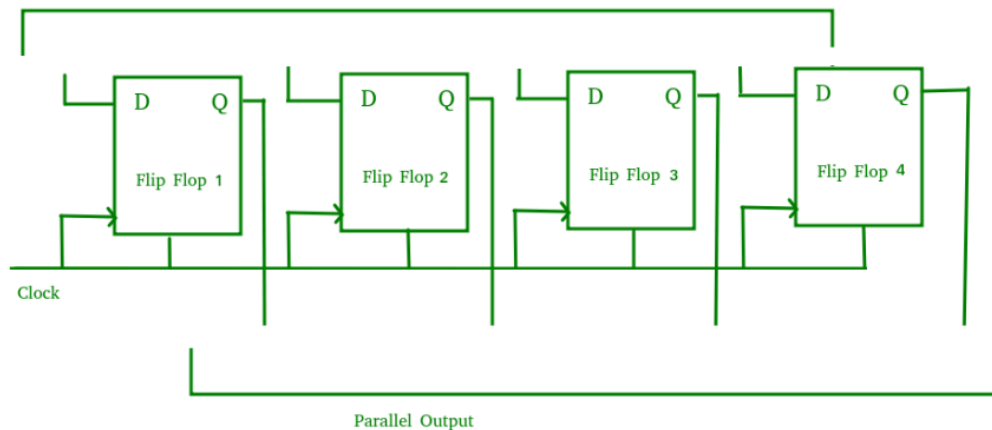


While taking the data there was a loop occurring in the memory, so a separate D flip-flop was added so that the new value gets latched and held until a clock pulse triggers an edge. Only when this trigger occurs the new value is sent to the board. This prevents looping, eliminates the chance of bugs, and makes debugging easier. Even if someone makes a mistake—whether a single or multiple errors—we can always revert to the previous state. As soon as we trigger it, the board updates.

PIPO Shift register:

A PIPO shift register is a collection of flip-flops arranged in a series, with each flip-flop capable of storing one bit of data. The primary characteristic that distinguishes a PIPO shift register is its ability to load data and output it in parallel.

PIPO Shift Register Circuit Diagram



In the given memory we used PIPO. Because we want to load the data parallelly and get the output parallelly.

Like in the given circuit diagram there is a control pin and a memory pin. After doing all the calculations like adding ladder values or subtracting snake value we now had to store the value so that we can use this value in the next run. When a move is done the last value goes to the memory pin and using a clock pulse the value is saved using the shift register. The main purpose of the control pin is to control whether to turn off the memory or change the previous memory values to new ones. When we are in player 1 the value entered is saved (control pin is 1) after switching we go to player 2 then the memory of the player 1 is disabled. So the new value is added to player 2. After the first run a new value comes and switches the previous value and holds the current value and so on.

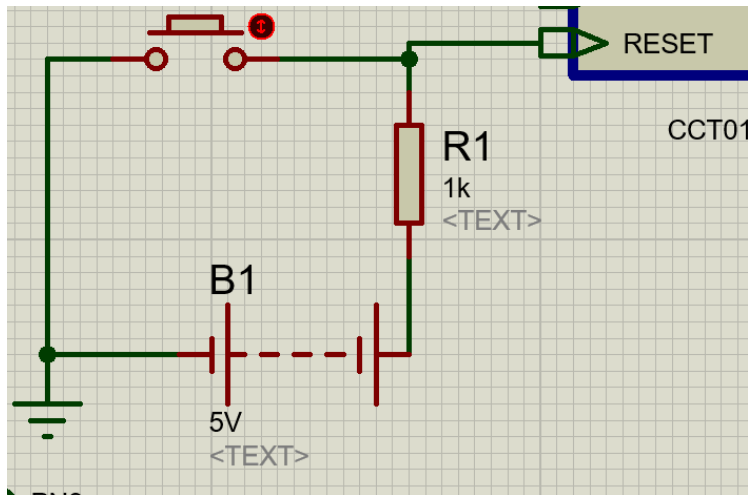
That is how the memory module works in our snake and ladder game.

Trigger:

There was a loop occurring, so we added a separate D flip-flop. This ensures that the new value gets latched and held until a clock pulse triggers an edge. Only when this trigger occurs does the new value get sent to the board. This prevents looping, eliminates the chance of bugs, and makes debugging easier. Even if someone makes a mistake—whether a single or multiple errors—we can always revert to the previous state. As soon as we trigger it, the board updates.

Reset Module:

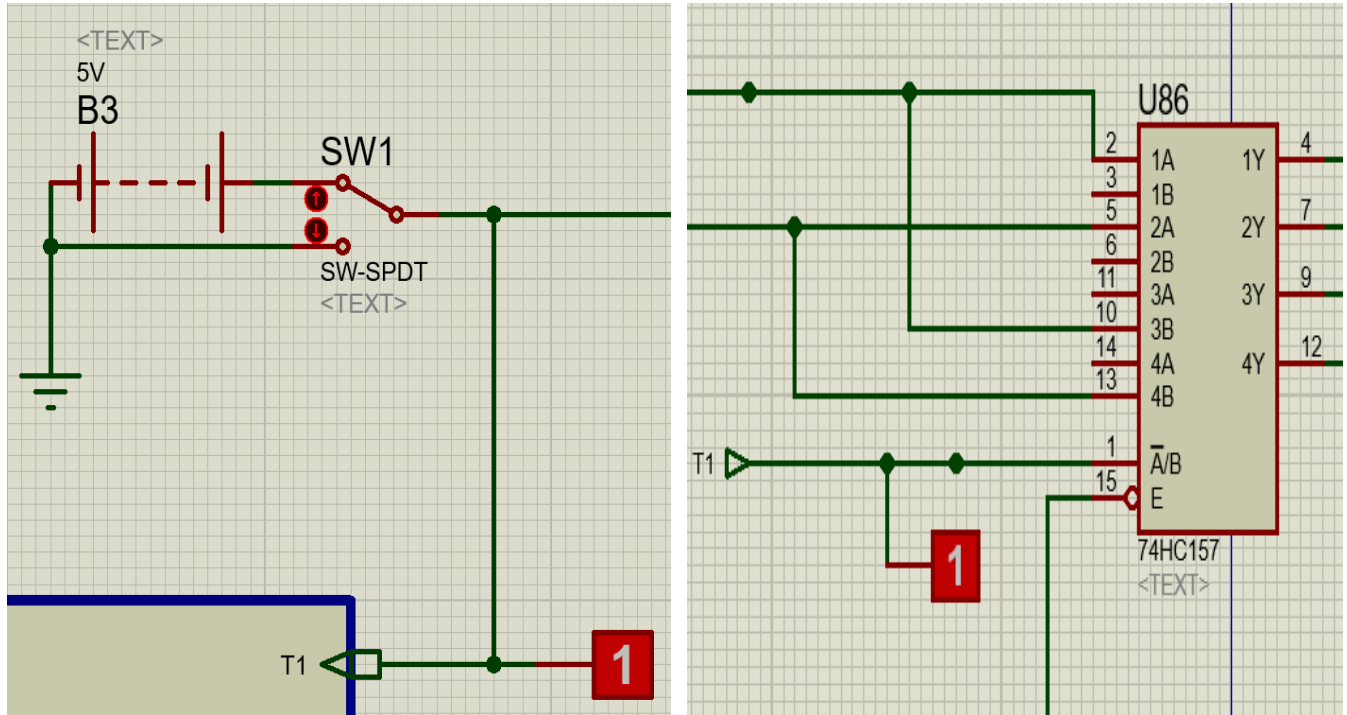
There is a reset button attached to the memory subcircuit, a press of which reset the player positions to zero, restarting the game. A press of the reset button utilizes the “Clear” input of the IC 7474 D flip-flop. The clear input disregards the existing inputs and outputs, forcing the player positions to reset to zero. The clear pin on the IC 7474 is in active low condition. There is also a pull up resistor connected to the battery. The pull up resistor keeps the voltage at a stable high level and takes it to zero when the reset button is pressed.



Player Selector Module:

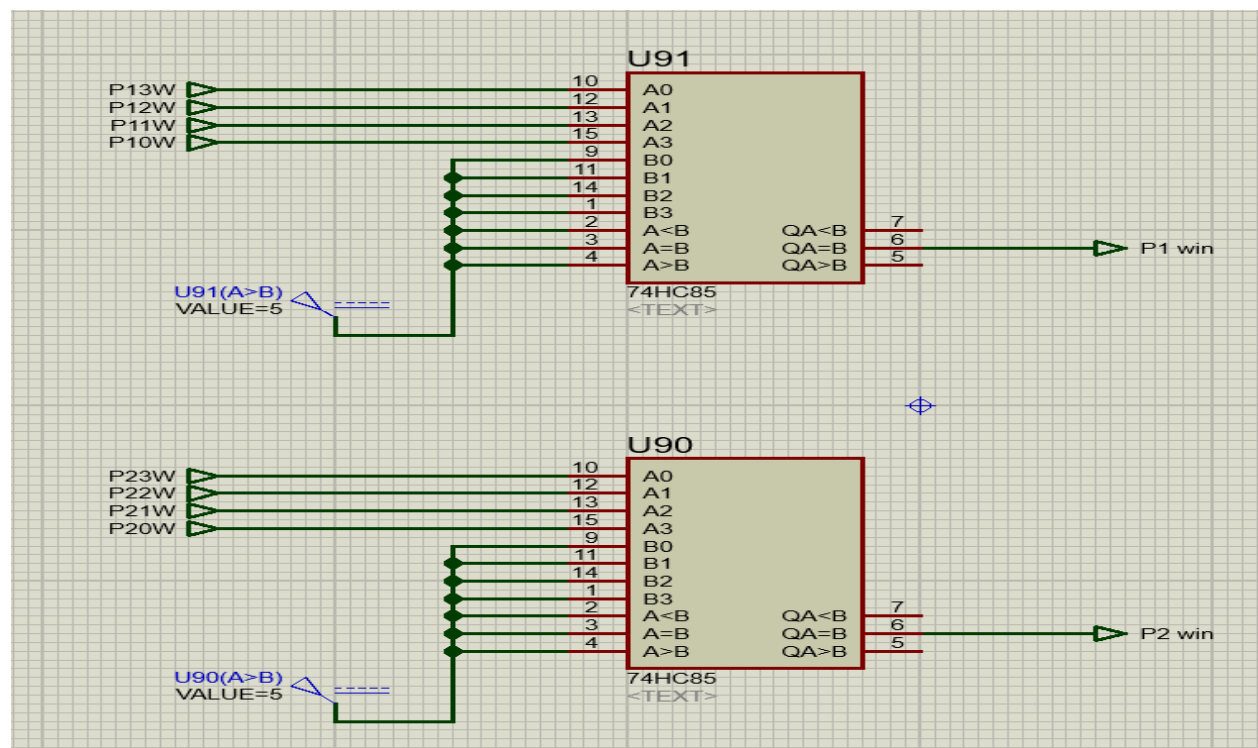
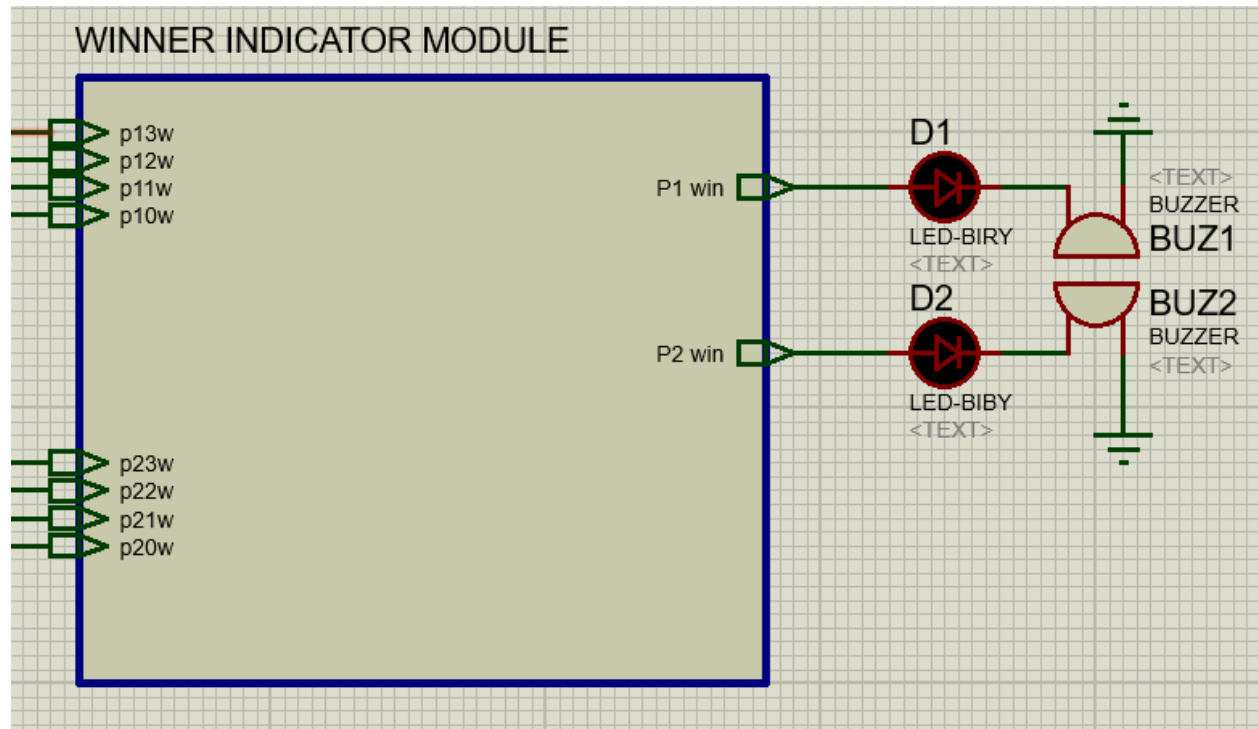
Right above the dice module is a switch for switching the current player. The switch is attached to a battery and takes an input to the dice module

subcircuit. An IC 74HC157 multiplexer is then used to select between the players. The 0 or low condition indicates that the Player 1 is chosen while 1 or high condition refers to the Player 2.



Winner Indicator Module:

The winner indicator module incorporates two 4-bit magnitude comparators using the IC74HC85. Two separate comparators are in use for both the players. The comparators compare the player positions with the value of binary 1111, the last position on the board. When a player is in the last position, is declared the winner. Two LEDs and Buzzers are used to indicate the win. Whichever player wins, their LED is lit and the buzzer rings.



Complete Integration of Digital Snake Ladder Game:

After putting the winner indicator module, the digital snake ladder game is now functioning with all its components and features. Players take turns to make their move using the dice helped by a 555 timer IC. The generated number shown on the 7-segment display is then forwarded to the memory and to the player position decoders for the designated LEDs to be lit. If a player encounters a snake or a ladder then the player's position is shifted accordingly using adders or subtractors. The player selector switch is pressed to select the next player to take turn. Ultimately when a player reaches the position 15, binary 1111, the winner indicator module compares it with the winning condition to declare the winner, shown with an LED and a buzzer. The reset button is then pressed to restart the game. The reset button can however be pressed at any time during the game to start over from position 0.

Problems Faced:

Several problems were met with during the development of the game and were handled cautiously. The memory falls into a loop where the input is passed continuously until a player wins. This problem was solved by adding another set of D flip-flops and we used a manual trigger to control the output. The reset button was not working for both players at once. When we pressed the reset button, only the player selected was reset.

Discussions:

Upon completion of the phases C, D, and E, the digital snake and ladder game is now ready to play. The outcome is a successful implementation of various electronic components to compile the project. Although we faced several challenges throughout the project, we were able to handle them

efficiently. There is a still problem with the reset button and the memory trigger.

Contributions:

ID	Name	Contribution
220021346	Musfique Ullah Khan Siam	Invalid Cases, Winning Module, Supported at Dice Module
220021348	Fasahat Labib	Memory, Winning Module, Dice Module
220021332	Fardin Ahmed Chowdhury	Invalid Cases, Debugging
220021345	MD.Mahir Rahman	Report Writing, Wiring
220021344	Kazi Mokabbir Hossain	Memory, Implemented the logic for Player 2 , Debugging
220021327	Kazi Adit Raiyan	Reset, player switch logic, supported at winning module, report writing

Additionally, all the members contributed to the project with meaningful insights, theoretical knowledge and application in sessions conducted both offline and online.